

Ordenação

Muitos problemas de programação competitiva envolvem ordenação, sendo muito comum em questões de busca. Além disso, muitas questões envolvem uma solução gulosa, onde construímos uma solução escolhendo sempre o maior ou menor elemento que estiver disponível naquele momento.

Função `sort`

A linguagem C++ possui uma função pronta para ordenar coleções de elementos (arrays, vectors, strings etc), chamada `sort`. Essa ordenação é realizada com complexidade de $O(n \log n)$, utilizando um mix de algoritmos eficientes como o quicksort.

A função requer um ponteiro para o início da coleção e um ponteiro para o final da coleção. Para utilizar `sort` com um array estático, basta chamar a função passando o vetor e o vetor + tamanho:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int V[5] = {7, 3, 15, 1, 9};
    int n = 5;
    sort(V, V + n);
    for(int i = 0; i < n; i++){
        cout << V[i] << " ";
    }
    // imprime 1 3 7 9 15
    cout << endl;
}
```

Também pode ser utilizada com vector, passando o `V.begin()` e `V.end ()`:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    vector<int> V = {7, 3, 15, 1, 9};
    sort(V.begin(), V.end());
    for(int i = 0; i < V.size(); i++){
        cout << V[i] << " ";
    }
    // imprime 1 3 7 9 15
    cout << endl;
}
```

Ordenação customizada

Por padrão, a função `sort` ordena os elementos em ordem crescente. Mas é possível escolher o critério de ordenação.

Para isso, precisamos implementar uma função de comparação que deve retornar `true` se o primeiro elemento deveria vir antes do segundo elemento, e `false` caso contrário. Depois de implementar esse comparador, basta passar como terceiro parâmetro da função `sort`.

Considere o exemplo de ordenação decrescente:

```
#include <bits/stdc++.h>
using namespace std;

bool compara(int a, int b){
    return a > b;
}

int main(){
    int V[5] = {7, 3, 15, 1, 9};
    int n = 5;
    sort(V, V + n, compara);
    for(int i = 0; i < n; i++){
        cout << V[i] << " ";
    }
    // imprime 15 9 7 3 1
    cout << endl;
}
```

Essa função é bem útil quando precisamos ordenar structs de acordo com algum dos campos específicos. Por exemplo, considere ordenar uma lista de pessoas de acordo com a idade decrescente e em caso de empate, pelo nome em ordem alfabética:

```
#include <bits/stdc++.h>
using namespace std;

struct Pessoa{
    string nome;
    int idade;
};

bool compara(Pessoa a, Pessoa b){
    if(a.idade == b.idade) return a.nome < b.nome;

    return a.idade < b.idade;
}

int main(){
    Pessoa pessoas[4] = {"zeno", 35}, {"joao", 20}, {"abner", 35}, {"ana", 20};
    int n = 5;
    sort(pessoas, pessoas + n, compara);
    for(int i = 0; i < n; i++){
        cout << pessoas[i].idade << " " << pessoas[i].nome << endl;
    }
    // imprime:
```

```
// 35 abner  
// 35 zeno  
// 20 ana  
// 20 joao  
}
```

falta mais algum detalhe do sort? complete: falta explicar que a função de comparação pode ser uma função lambda, e que também é possível usar `greater<int>()` para ordenação decrescente sem precisar criar uma função de comparação. também falta